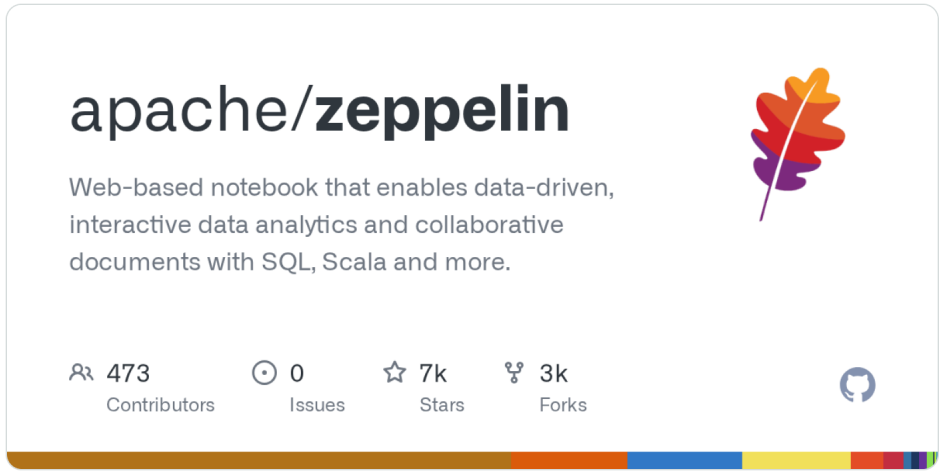
AWSKRUG 프론트엔드 소모임

통과하는 E2E 테스트를 믿어도 될까

이용재 - Web FE 개발자
LinkedIn/GitHub: @voidmatcha
SNOW

문제 인식

2025 OSSCA Zeppelin 멘티로 참여



E2E 환경 밑바닥부터 직접 구축

테스트는 LLM으로 작성

(Claude Opus 4.1 기준)

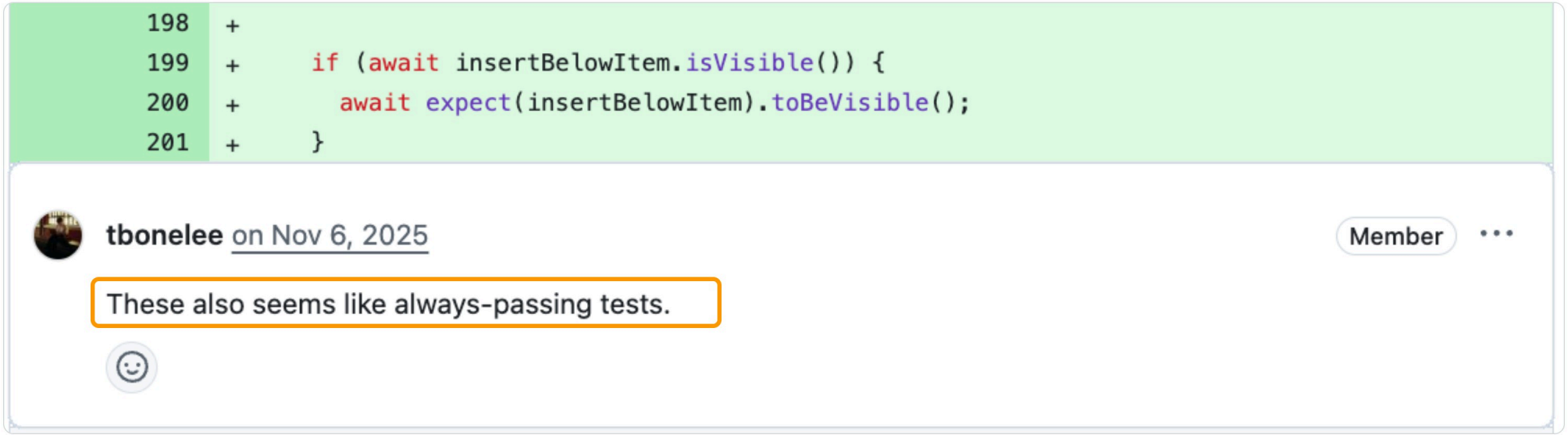
LLM은 생각보다 많이 틀림

일반적인 개발 원칙(YAGNI, DRY 등) 위반 잦음

통과만 하고 검증하지 않는 테스트 생성

직접 받은 리뷰

PR 리뷰를 받으며 깨닫게 된 "항상 통과하는 테스트"



```
198 +
199 +     if (await insertBelowItem.isVisible()) {
200 +       await expect(insertBelowItem).toBeVisible();
201 +     }
```

tbonelee on Nov 6, 2025 — Member

These also seems like always-passing tests.

실제로 merge된 코드

이 테스트는 언제 실패할까요?

calcom/cal.diy / PR #28486 (merged)

```
await expect(page.getByTestId("away-emoji")).toBeTruthy();
```

화면에 away-emoji가 없으면 실패할까요?

답: 실패하지 않음. Locator 객체는 항상 존재

왜 통과할까

element가 없어도 통과

```
$ npx playwright test

✓  1 away emoji - before (toBeTruthy) (83ms)
✘  2 away emoji - after (toBeVisible) (2.1s)
   ⋮
   Error: element(s) not found
   ⋮
  1 failed
  1 passed (3.1s)
```

Locator는 element 자체가 아닌 "찾는 방법" / 객체는 항상 존재

비슷한 사례

`if (isVisible)`	`.catch(() ⇒ {})`	`test.only`	**Optimistic UI만 확인**

Playwright Locators / 공식 문서

다른 사람들도 비슷한 문제를 겪음

Kent Beck: 에이전트가 assertion과 테스트를 지움



Kent Beck

Extreme Programming 창시자
『Test-Driven Development: By Example』 저자

Now the complexity has gotten to be too much for the genie. There's no path consisting only of tiny steps getting from here to there.

The genie doesn't respond well to this situation. I've seen infinite loops. I've seen truly pernicious behavior like deleting assertions from tests, deleting whole tests, & faking large swathes of implementation. These behaviors may allow the genie to reward itself with a biscuit, but they destroy trust, my trust. And just when I was starting to have fun programming again...

LLM 스킬 개념이 막 나오던 시점

그래서 스킬로 만들기 시작

voidmatcha/**e2e-skills**



Playwright and Cypress skills for Claude Code and Codex: generate tests, catch false-green tests and test smells, debug failures.

3 Contributors · 2 Issues · 21 Stars · 3 Forks

github.com/voidmatcha/e2e-skills

e2e-skills 첫 단계

Best Practices부터 조사

공식 문서

Playwright / Cypress 가이드

직접 겪은 사례

Zeppelin 등에서 재현된 이슈

Playwright Best Practices / Cypress Best Practices

1단계 결과 / 심각도별 24개 룰

이렇게 추가된 24개 룰

P0 / 항상 통과

제품이 깨져도 테스트가 통과

#1 이름과 assertion 불일치
#2 마지막 상태 검증 없음
#3 에러 삼키기 (#3b Cypress)
#4 의미 없는 assertion
#5 우회 조건문
#7 test.only 남김
#8 assertion 누락
#12 인증 setup 누락

P1 / 원인 파악 어려움

실패해도 무엇이 문제인지 불명확

#6 raw DOM 쿼리
#9 고정 sleep
#10 flaky 패턴
#13 POM 불일치
#14 하드코딩 계정
#15 expect await 누락
#16 action await 누락
#17 Page 직접 selector
#18 expect.soft 누수
#19 모듈 전역 상태
#20 실제 API를 통한 데이터 변경
#22 요청 확인 없는 Optimistic UI

P2 / 유지보수

쌓이면 비용이 됨

#11 YAGNI · 좀비 spec
#21 수동 세션 파일 의존
#23 렌더 조건 무시 fixture

LLM만 쓰던 구조에서 lint + LLM 구조로

LLM에게 전부 맡기면 false positive가 많음

0.39

정밀도 (규칙 없이 LLM만)

27건

반복 실행에서 안정된 예측의 false positive

lint가 후보를 거르고 LLM이 판단하는 lint + LLM 구조로 전환

36개 케이스 × 5회 / gpt-6-sol / 개발 단계(공개 케이스)

lint를 적용하다 보니…

필요한 규칙을 eslint-plugin-playwright에 추가

mskelton/eslint-plugin-playwright #470

`no-unnecessary-assertions`

화면에 없어도 통과하는 assertion을 toBeVisible()로 교체

2026-07-21 merged / v2.11 recommended

lint 다음 단계 / AST 기반 scanner

scanner로 후보 선별, LLM 검토로 확정

```
$ scan.sh tests/
[AST][LLM-TRIAGE] sg-4f-locator-as-truthy (1 hit;
Playwright expect provenance unproven)
  …/tests/dashboard.spec.ts:5:9
```

Tier 1	Tier 2	Tier 3
ESLint	ast-grep	rg + Python

실제로 실패하는지 검증

"통과했는가?"가 아니라 "실패할 수 있는가?"

scanner
후보 선별 / reviewer

→

LLM 검토
반증 / false positive 제외

→

실행 검증
e2e-reviewer opt-in /
V2·V3 probe

실패부터 확인

Willison: 실패해 본 적 없는 테스트는 신뢰할 수 없음



Simon Willison

Datasette
Agentic Engineering Patterns
가이드 저자

It's important to confirm that the tests fail before implementing the code to make them pass. If you skip that step you risk building a test that passes already, hence failing to exercise and confirm your new implementation.

That's what "red/green" means: the red phase watches the tests fail, then the green phase confirms that they now pass.

Red/green TDD / Agentic Engineering Patterns / simonwillison.net

결함을 넣어 확인하기 (e2e-reviewer opt-in)

assertion을 negate하고 제품에 결함을 넣어 실패 확인

V2: assertion negate

```
// 원본
await expect(status).toHaveText("Count: 1");
// negate한 복사본
await expect(status).not.toHaveText("Count: 1");
```

V3: 제품 동작에 결함 주입

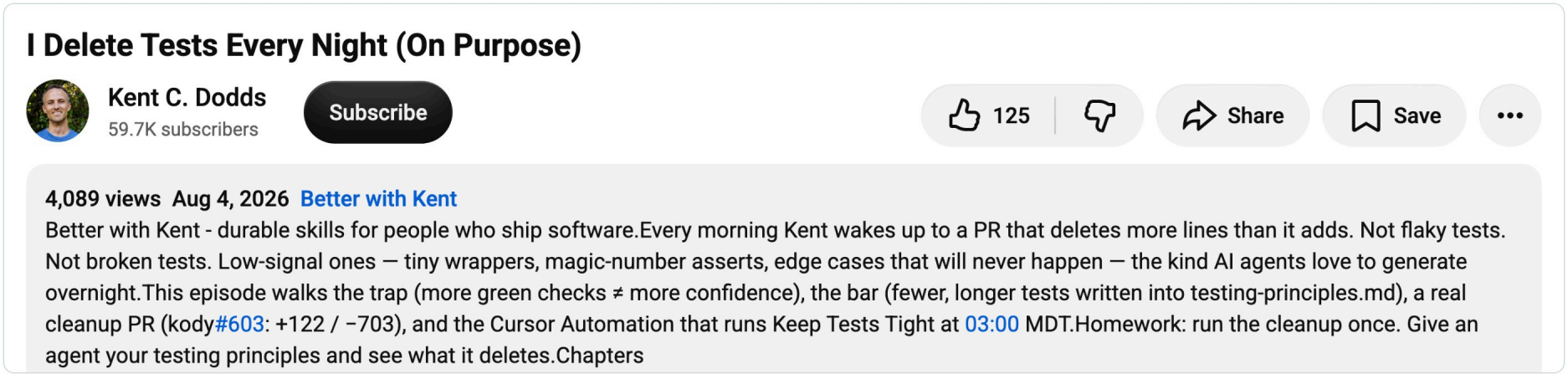
```
// 장애: count 응답을 0으로
await page.route("**/count", r ⇒
  r.fulfill({ json: { count: 0 } }));
// assertion은 그대로, 여기서 실패해야 통과
await expect(status).toHaveText("Count: 1");
```

여기서 다시 질문

요즘 모델이 좋아졌다는데 여전히 필요할까요?

최근에도 나오는 비슷한 주장들

Kent C. Dodds: 에이전트가 매일밤 신뢰할 수 없는 테스트를 생성



최근에도 나오는 비슷한 주장들

Thoughtworks: 항상 통과하는 테스트는 mutation testing 적용

thoughtworks

Technology Radar
기법 / Trial 단계
2026-04-15

With AI-generated test cases now commonplace, mutation testing acts as a reinforcement layer for catching "perpetually green" tests — those that pass regardless of logic changes due to missing assertions or decoupled mocks. By using tools such as Stryker, Pitest or cargo-mutants, we shift the focus from how much code is executed to how much code is actually verified, particularly in core domain logic. The goal is to ensure that a passing test suite is a reliable signal of functional correctness, rather than simply a report of which lines were executed.

추가적인 근거들 (1/3) / 최신순

실무자분들의 글과 발표

출처	핵심
Paul Stack (Swamp) / 2026-09-23	에이전트가 쓴 테스트는 검증이 아니라 명세
Michael Bromley / 2026-09-07	tautological 테스트, 약한 matcher, 조건부 assertion을 분류
토스 FE 웨비나 / 2026-09-06	AI로 만든 결과물을 어떻게 검증하는가를 다룬 세션
Tariq Shaukat (Sonar) / 2026-07-21	병목이 생성에서 검증으로 옮겨 갔다는 발표
Jeff Thoensen, DEV / 2026-07-14	화면 메시지만 확인한 비밀번호 재설정 테스트 / 실제 버그 확인
NAVER D2 / 2026-06-15	Playwright E2E 하네스로 에이전트가 짠 코드를 어떻게 믿을지 다룬 세션
Tian Pan / 2026-05-17	에이전트가 코드를 보고 쓴 테스트는 첫 실행부터 통과 / assertion laundering
Dave Farley, Aviator / 2026-02-13	AI 때문에 검증이 병목이 되는 이유

추가적인 근거들 (2/3) / 최신순

관련 측정과 사례

출처	핵심
Ivan Davidov, YouTube / 2026-09-09	같은 프롬프트로 Claude Code를 4번 실행, 한쪽에서 false positive assertion
Malhotra, SD Times / 2026-09-02	AI로 수정 뒤 테스트가 잘못된 요소를 확인하면서 green으로 돌아갈 수 있음
Szymon Fiedler, RBQ 2026 / 2026-08-11	테스트가 통과해도 실제로 검증하는지 묻는 mutation testing 발표
Paul Newell, DEV / 2026-06-30	AI가 쓴 테스트 20개가 모두 통과했지만 mutant 95개 중 27개가 살아남음
Lee, arXiv / 2026-05-02	자동 테스트 수리에서 assertion 약화와 테스트 삭제가 수렴의 우회 수단으로 쓰임
Gleb Bahmutov (Mercari US) / 2026-04-28	AI가 만든 assertion은 body가 보이는지만 확인
Senko Rašić / 2026-03-22	커버리지 98%인데 LLM이 제안한 mutant 40개 중 30개만 잡힘(75%)
David R. MacIver, Antithesis / 2025-12-10	LLM이 쓴 단위 테스트도 정확성 가드레일로 부족, 속성 기반 테스트 필요

추가적인 근거들 (3/3) / 최신순

연구, 공식 문서, 설문

출처	핵심
Vitest 가이드 / 2026-10-12 캡처	호출만 하고 throw 여부만 보는 테스트는 false confidence라고 경고
Currents / 2026-08-27	Healer가 locator를 넓혀 실패할 수 없게 함
Kaufman 외, arXiv / 2026-07-09	프로그래머가 틀린 assertion을 알아본 비율은 49%(86명)
Teoh 외, WebTestPilot / 2026-02-12	웹앱 4개, 주입 버그 100개에서 검출 정밀도와 재현율 96%
Sonar 설문 / 2026-01-08	개발자 96%가 AI 코드 정확성을 완전히 믿지 않고, 커밋 전 항상 확인은 48%
Mathews 외, arXiv / 2024-12-18	LLM 테스트 생성기의 설계 선택이 버그 발견을 막는다는 논문
Alshahwan 외, Meta TestGen-LLM / 2024-02-14	생성된 테스트 중 25%만 커버리지를 늘렸고, 결함 검출률은 아님
Anthropic, Best practices for Claude Code	실행할 수 있는 검사가 없으면 '끝난 것 같다'가 유일한 신호

결론

모델이 좋아져도 문제는 남음

PASS ≠ 검증

Locator는 항상 존재

healer 단계의 모호함

테스트 탓인지 product 탓인지 구분 못 함

병목은 검증

생성보다 확인이 어려움

"PASS"가 곧 "동작"이어야 믿을 수 있다

다시 돌아와서

voidmatcha/e2e-skills 소개

`playwright-test-generator`

`e2e-reviewer`

`playwright-debugger`

`cypress-debugger`

github.com/voidmatcha/e2e-skills / skills

e2e-reviewer - benchmark

잘못된 판단이 27건에서 2건으로

스킬 전체

0.95

정밀도

잘못된 판단 2건

찾은 결함 100%

패턴 목록만

0.89

정밀도

잘못된 판단 5건

찾은 결함 100%

규칙 없음

0.39

정밀도

잘못된 판단 27건

찾은 결함 43%

F1 차이	기준	결과 (95% 신뢰구간)	판정
전체 대 규칙 없음	+0.3 이상, 하한 0 초과	+0.571 (+0.411 ~ +0.738)	**통과**
전체 대 목록만	판정 기준 없음	+0.034 (0.000 ~ +0.072)	참고용

36개 케이스 × 5회 / gpt-6-sol / 개발 단계(공개 케이스)

필드테스트 진행 / merge됨

총 15개 오픈소스 PR merge

```
203   -     await expect(page.getByTestId("away-emoji")).toBeTruthy();
      203   +     await expect(page.getByTestId("away-emoji").first()).toBeVisible();
```

Cal.com #28486 2026-03-25 merged


★ 91.2k


★ 79.6k


★ 73.3k


★ 55.5k



★ 49.0k


★ 47.5k


★ 22.1k


★ 20.8k


★ 13.6k


★ 11.2k


★ 9.5k


★ 7.4k

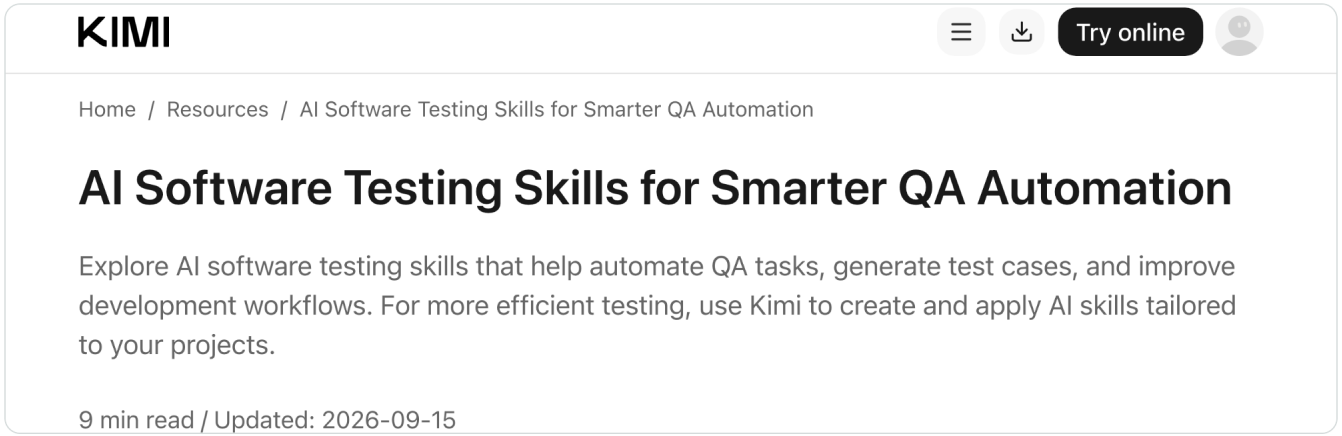

★ 6.7k


★ 5.9k


★ 2.7k

외부 소개

Kimi 공식 사이트의 테스트 스킬 소개 페이지에 수록



e2e-skills	AI agent testing toolkit for Playwright and Cypress: generate E2E tests from scratch, review existing specs against 24 anti-patterns (P0/P1/P2 silent-always-pass smells), and debug flaky failures from playwright-report/ or cypress/reports/.	https://github.com/voidmatcha/e2e-skills

kimi.ai/resources/software-testing-skills

v3.0.0

이번 발표를 준비하면서 개선한 것

generator	6.74배 느림 / 결함 검출 미입증	→	**공식 에이전트에 위임, 규칙과 scanner만 추가**
debugger	입력 7~10배 / 이득 없음	→	**스킬 축소 / 입력 0.69배 / 정확도 비슷**

v3.0.0

현재 generator 구조와 흐름



Playwright Test Agents / Playwright 1.56 이상

로드맵

앞으로 더 개선하고 싶은 것

설치와 실행을 쉽게 하고, 검토 효과와 비용 확인하기

개발 중

scanner를 독립 도구로

Node 기반 CLI와 npm 패키징 준비

기존 scanner와 결과 및 환경 호환성 검증

후속 검증

리뷰 효과와 비용 개선

lint와 일반 리뷰 대비 추가 효과 확인

지침과 참조를 줄이고 탐지 성능 유지 확인

오늘 가져갈 질문

LLM이 짠 테스트에
안심하고 계셨나요?

감사합니다